ATIVIDADE PRÁTICA AULA 1 · PROF. FRANCISCO FAMBRINI

Interfaces Web/Mobile para Coleta de Dados: Gradio x Streamlit

Aluno: Matheus Neto · **Data de entrega:** 01/11/2026

1. Links da entrega

Repositório GitHub	https://github.com/Matheus22sn/cadastro-pacientes-gradio-streamlit
App no Streamlit Community Cloud	https://cadastro-pacientes-gradio-app-nqc6euceguzguhl3caxlkf.streamlit.app/

2. Estrutura do repositório

```
cadastro-pacientes-gradio-streamlit/
├── gradio_app/
│   ├── app.py               versão Gradio (execução local)
│   ├── pacientes.csv        arquivo de exemplo gerado nos testes
│   └── print_*.png          capturas de tela da versão Gradio
├── streamlit_app/
│   ├── app.py               versão Streamlit (publicada na nuvem)
│   └── print_*.png          capturas de tela da versão Streamlit
├── requirements.txt         gradio, streamlit, pandas
└── README.md                instruções de instalação e execução
```

3. Parte 1: versão Gradio (execução local)

O formulário foi construído com `gr.Blocks()` em coluna única, para funcionar bem no celular. Cada clique em **Cadastrar** valida os dados e acrescenta uma linha em `pacientes.csv` (o arquivo é criado com cabeçalho na primeira execução), junto com a data/hora do cadastro.

- ✓ Componentes: `gr.Textbox` (nome), `gr.Number` (idade), `gr.Dropdown` (convênio), `gr.Slider` 1–5 (prioridade) e `gr.Textbox` multilinha (motivo).
- ✓ Gravação em `pacientes.csv` com `timestamp`; caminho do arquivo relativo ao próprio script.
- ✓ Mensagem de confirmação e tabela `gr.Dataframe` com os 5 últimos cadastros (mais recente primeiro), exibida já ao abrir a página.
- ✓ Validação: nome, idade (0 a 120), convênio e motivo obrigatórios, com aviso único (`gr.Warning`) e sem perder o que já foi digitado.
- ✓ Formulário limpo automaticamente após um cadastro bem-sucedido.

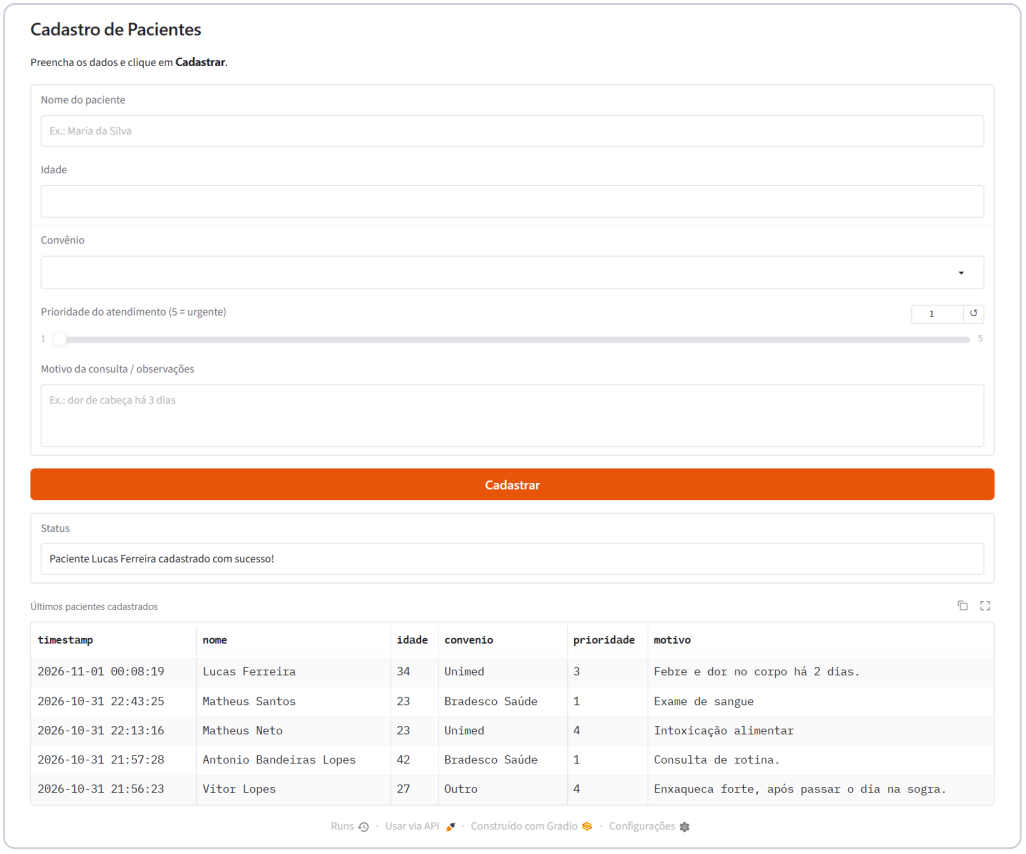


Figura 1 – Interface Gradio rodando localmente após um cadastro. (gradio_app/print_gradio_desktop.png)

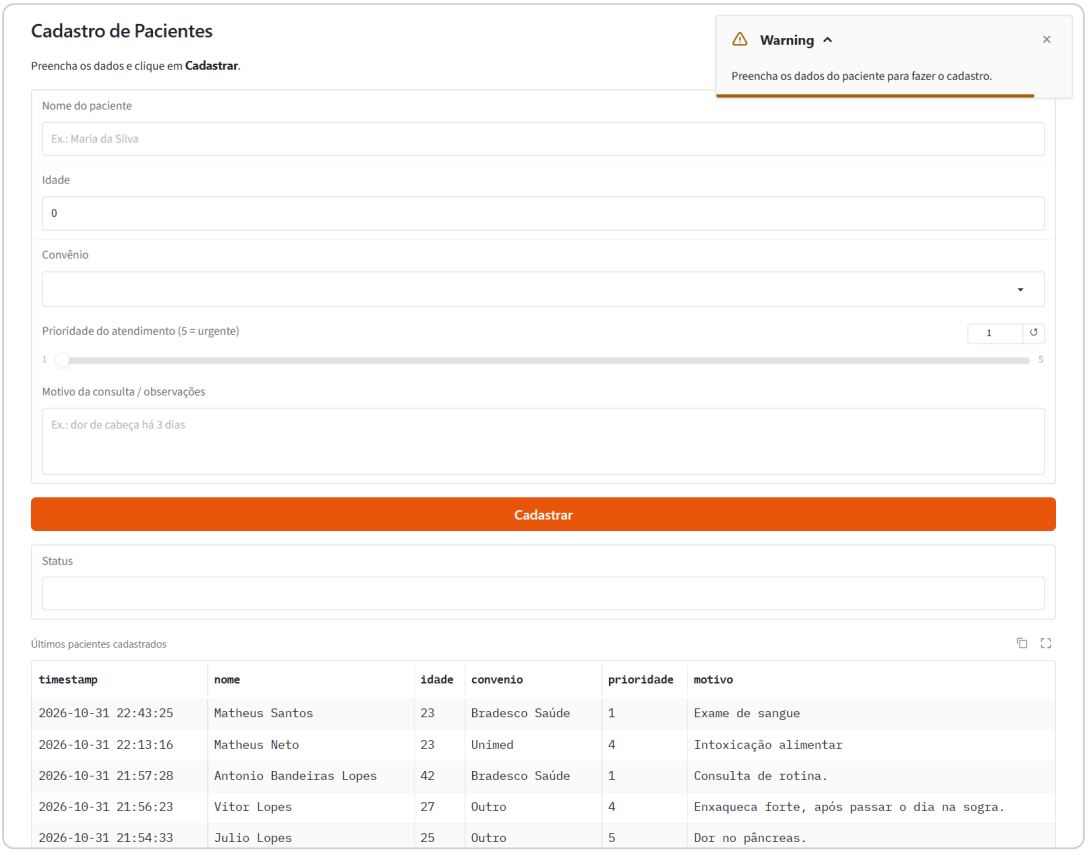


Figura 2 – Validação: tentativa de cadastro com o formulário vazio. (gradio_app/print_gradio_validacao.png)

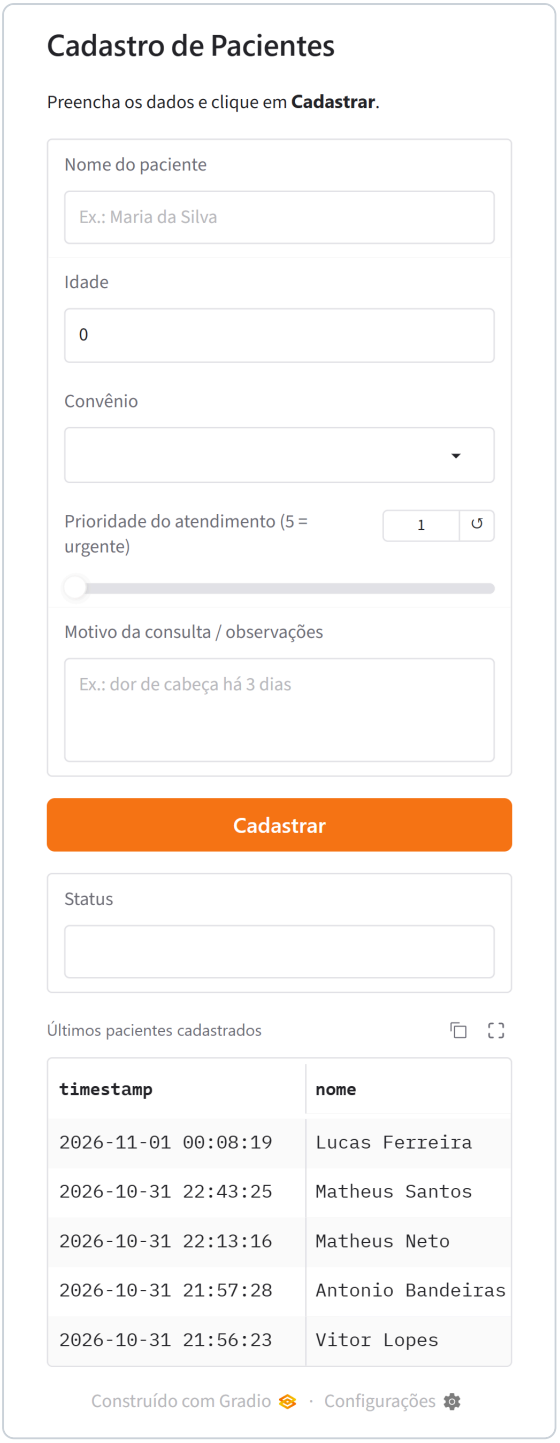


Figura 3 – Interface Gradio em tela de celular. (gradio_app/print_gradio_celular.png)

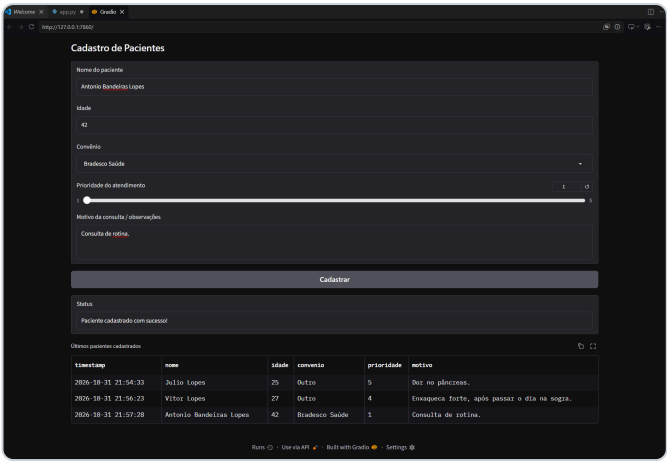


Figura 4 – Primeira versão rodando localmente (esqueleto da atividade). (gradio_app/print_gradio.png)

4. Parte 2: versão Streamlit (deploy na nuvem)

Os mesmos campos e a mesma lógica de gravação, agora com `st.form()`. Como o Streamlit reexecuta o script inteiro a cada interação, os pacientes da sessão ficam em `st.session_state`. O sistema de arquivos do Streamlit Community Cloud é efêmero, por isso há o botão **Baixar CSV** (`st.download_button`) para exportar os dados da sessão.

✓ `st.form()` agrupando os 5 campos; envio por callback (`on_click`), que limpa o formulário só quando o cadastro é válido.

- ✓ `st.session_state` acumulando os pacientes cadastrados durante a sessão.
- ✓ `st.download_button` para exportar o CSV da sessão.
- ✓ Validação com notificação única (`st.toast`).
- ✓ Data/hora gravada no fuso de São Paulo (o servidor do Streamlit Cloud usa UTC).
- ✓ Repositório público no GitHub com `app.py` e `requirements.txt` ; app publicado em share.streamlit.io a partir de `streamlit_app/app.py` .

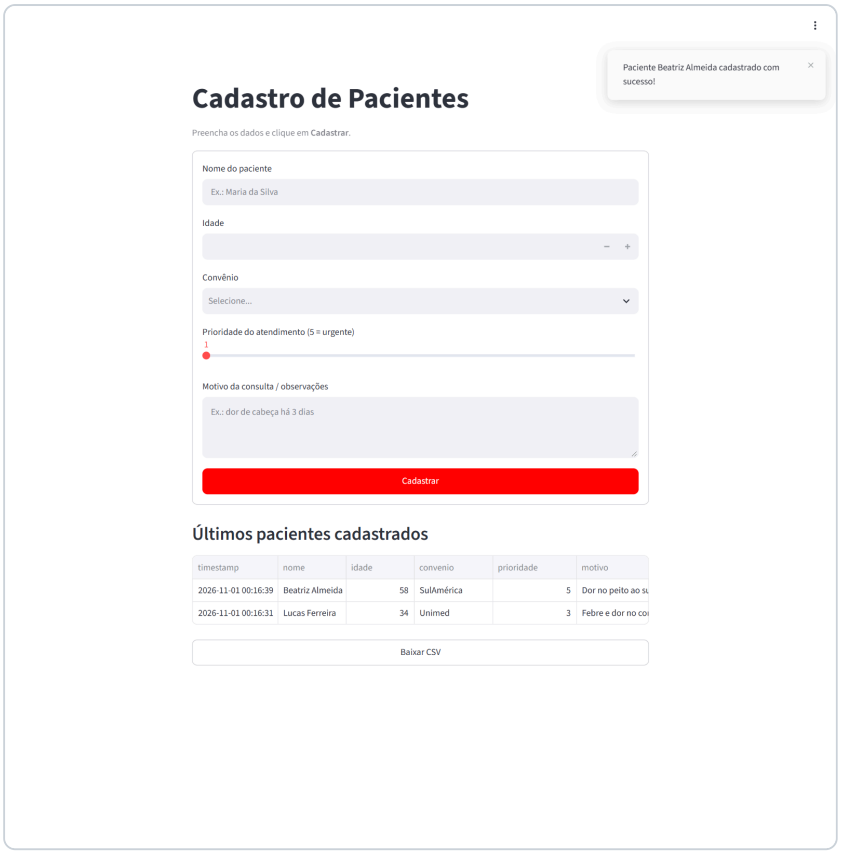


Figura 5 – App publicado no Streamlit Community Cloud após dois cadastros. (streamlit_app/print_streamlit_cloud.png)

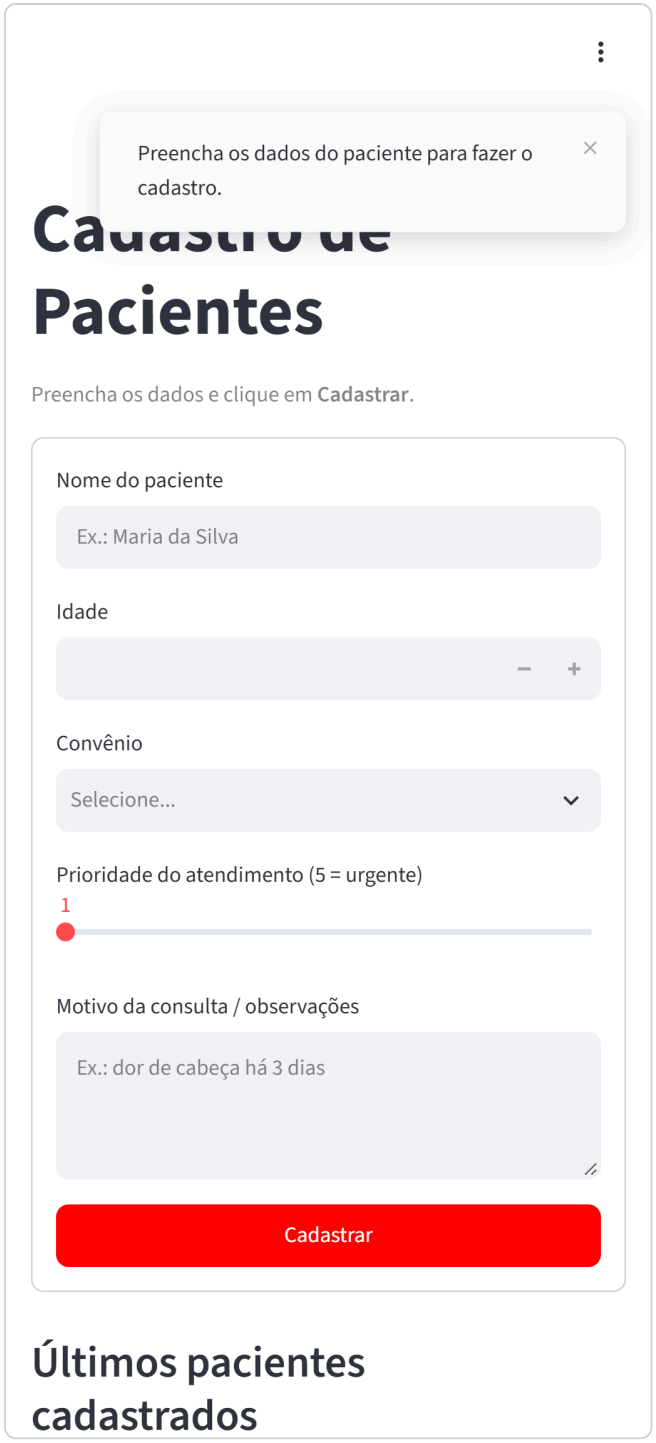


Figura 6 – App publicado, no celular: validação do formulário vazio. (streamlit_app/print_streamlit_celular_validacao.png)

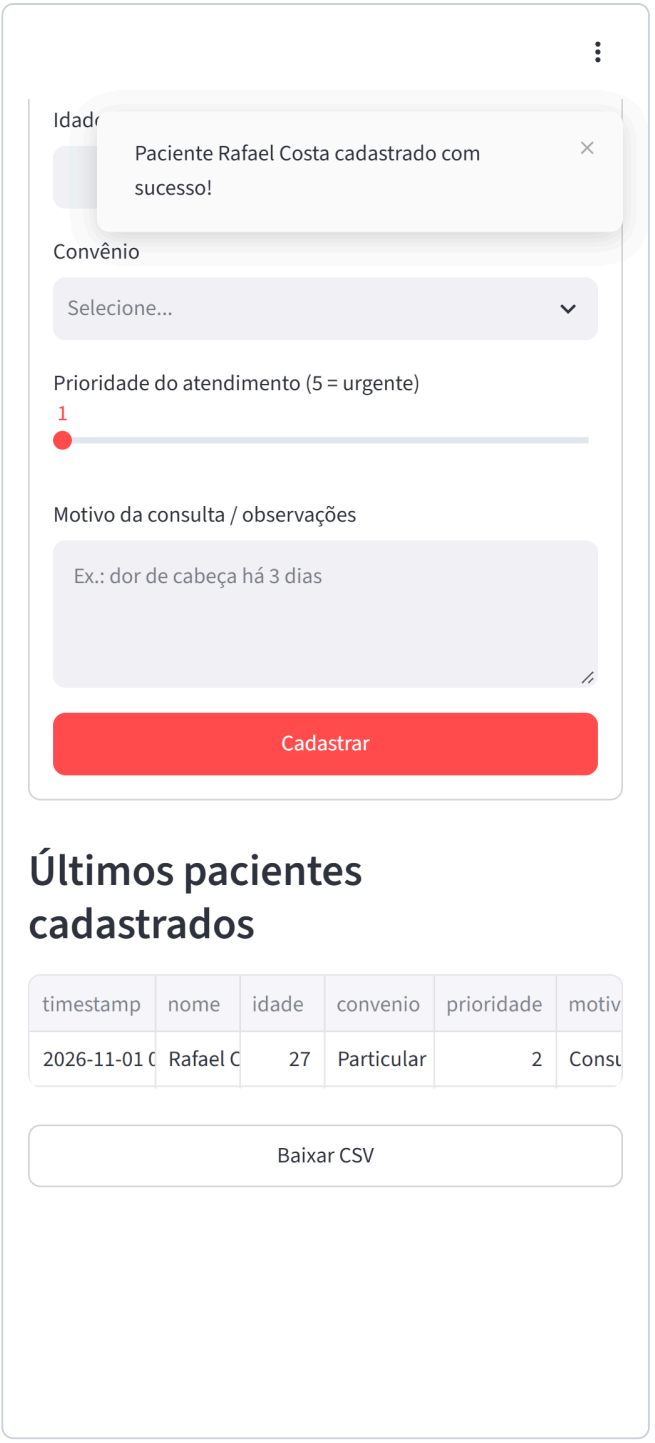


Figura 7 – App publicado, no celular: cadastro realizado e botão Baixar CSV. (streamlit_app/print_streamlit_celular.png)

5. Arquivo pacientes.csv gerado nos testes

Dados fictícios, gerados pela versão Gradio. O arquivo está em `gradio_app/pacientes.csv`.

timestamp	nome	idade	convenio	prioridade	motivo
2026-10-31 21:54:33	Julio Lopes	25	Outro	5	Dor no pâncreas.
2026-10-31 21:56:23	Vitor Lopes	27	Outro	4	Enxaqueca forte, após passar o dia na sogra.
2026-10-31 21:57:28	Antonio Bandeiras Lopes	42	Bradesco Saúde	1	Consulta de rotina.
2026-10-31 22:13:16	Matheus Neto	23	Unimed	4	Intoxicação alimentar
2026-10-31 22:43:25	Matheus Santos	23	Bradesco Saúde	1	Exame de sangue
2026-11-01 00:08:19	Lucas Ferreira	34	Unimed	3	Febre e dor no corpo há 2 dias.

6. Comparação: Gradio x Streamlit

	Gradio	Streamlit
Modelo de execução	A tela é montada uma vez; eventos (`.click`) chamam funções com `inputs` e `outputs`.	O script inteiro é reexecutado a cada interação.
Estado	Valores ficam nos componentes; a função recebe e devolve valores.	Variáveis comuns se perdem; o que precisa persistir vai em `st.session_state`.
Formulário	Componentes soltos dentro de `gr.Blocks`.	`st.form` agrupa os campos e só envia no clique.
Mensagens ao usuário	`gr.Warning` / `gr.Info`.	`st.toast`, `st.success`, `st.warning`.
Publicação	Link temporário com `share=True` ou hospedagem no Hugging Face Spaces.	Streamlit Community Cloud, conectado direto ao GitHub, com novo deploy a cada `git push`.
Persistência na nuvem	Nos dois casos, arquivo local não é confiável em hospedagem gratuita; para dados permanentes é preciso um banco de dados.	

Conclusão: o Gradio é mais direto para uma interface do tipo "entrada → função → saída", e o modelo de eventos deixa explícito o fluxo dos dados. O Streamlit exige entender a reexecução do script e o `session_state`, mas oferece mais recursos de página (tabelas, download, layout) e o deploy mais simples, direto do GitHub.

7. Como executar

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt

python gradio_app/app.py               # http://127.0.0.1:7860
streamlit run streamlit_app/app.py     # http://localhost:8501
```